Task VPC-AWS-EC2

Notes: Create a Jira ticket for this assignment, work on the tasks listed below, and submit the task completion document by EOD with clear steps and required evidence (outputs/screenshots)

Create VPC with 2 private and 2 public subnets.

Enable DNS Hostname in VPC.

Enable Auto Assign Public IP in 2 public subnets.

Add 2 private subnets in private route table.

Add 2 public subnets in public route table.

Public route table will have the routes to internet and local.

Create EC2 in public subnet with t2.micro and install PHP.

Configure NAT gateway in public subnet and connect to private instance.

Install Apache Tomcat in private EC2 and deploy a sample app.

Configure VPC flow logs and store the logs in S3 and CloudWatch.

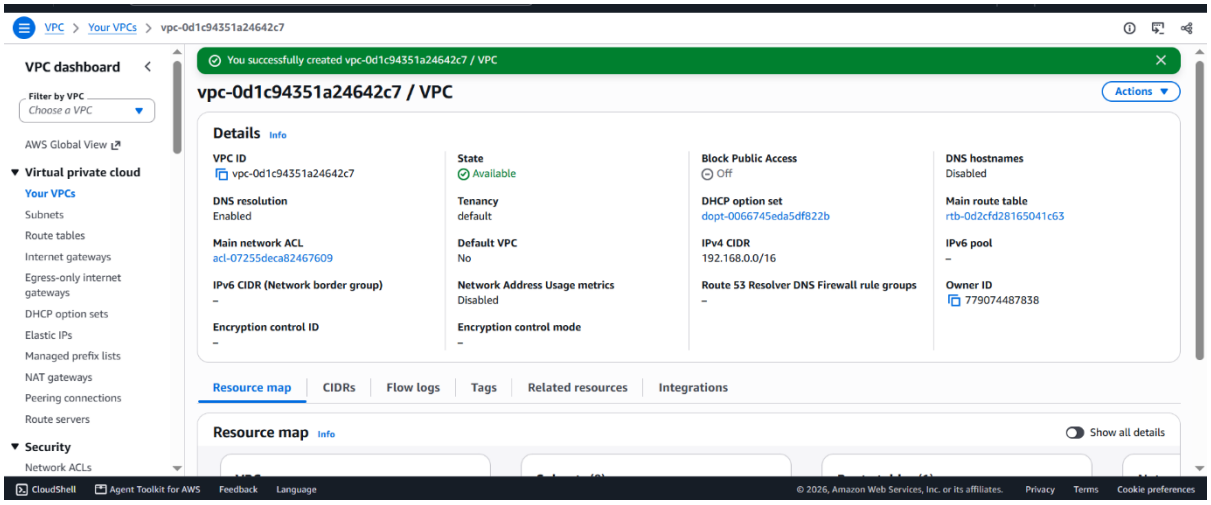
1) Create VPC with 2 private and 2 public subnets.

Step 1: Open the VPC service

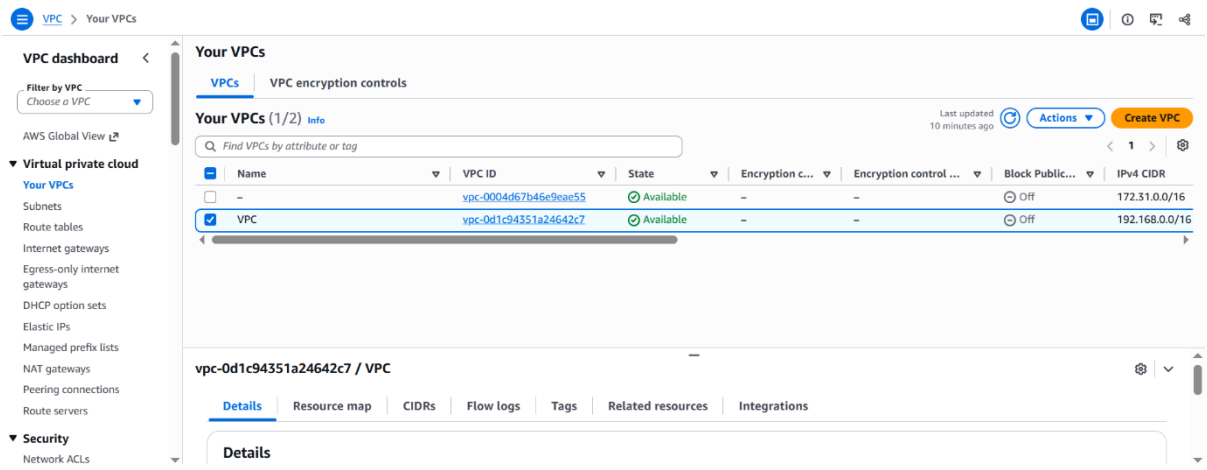
1. Sign in to your AWS Console.
2. In the search bar at the top, type **VPC**.
3. Click **VPC** under Services.
4. In the left-side menu, click **Your VPCs**.

Step 2: Create the VPC

1. Click **Create VPC**.
2. Under Resources to create, select **VPC only**.

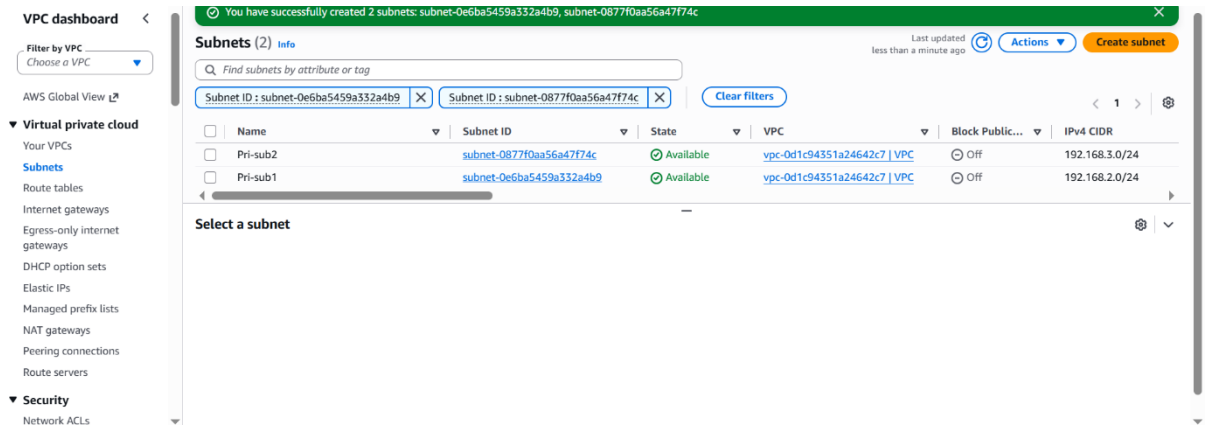


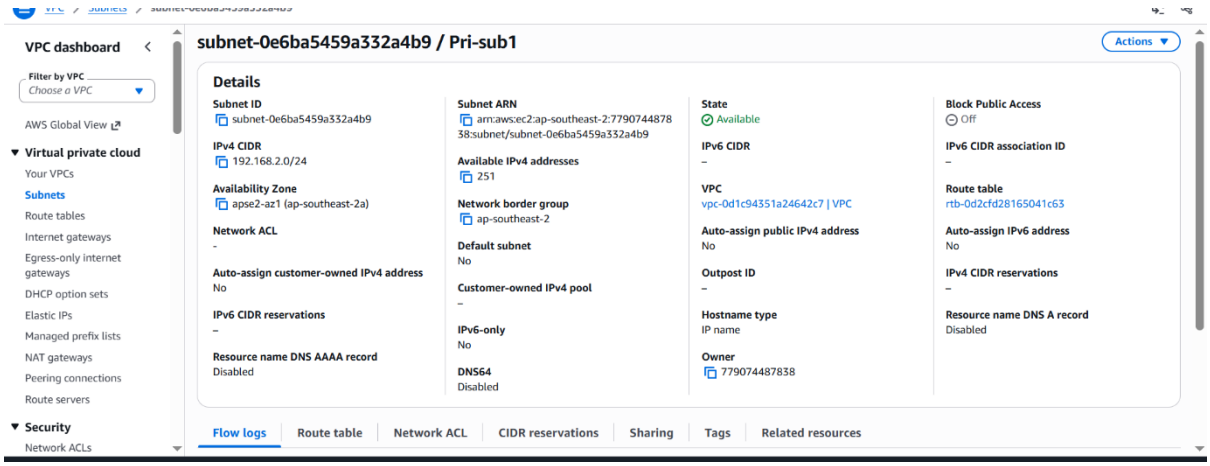
Setting	Value
Name tag	vpc
IPv4 CIDR block	192.168.0.0/16
IPv6 CIDR block	No IPv6 CIDR block
Tenancy	Default

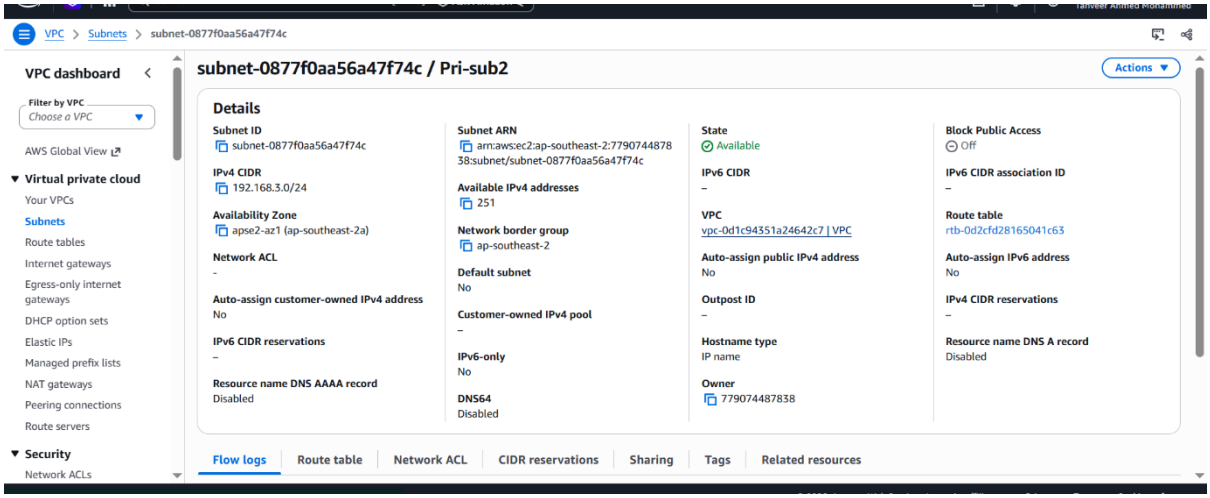


Now creating a Private Subnets.

- [ ] **Pri-sub1**
  - **Subnet ID:** subnet-0e6ba5459a332a4b9
  - **IPv4 CIDR:** 192.168.2.0/24
  - **State:** Available
  - **Block Public Access:** Off
- [ ] **Pri-sub2**
  - **Subnet ID:** subnet-0877f0aa56a47f74c
  - **IPv4 CIDR:** 192.168.3.0/24
  - **State:** Available
  - **Block Public Access:** Off

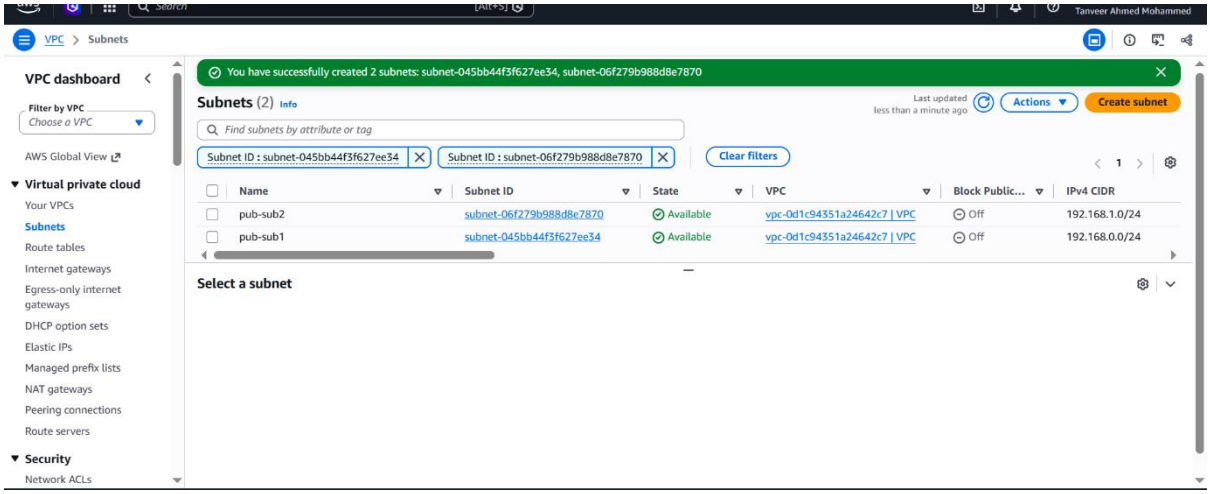


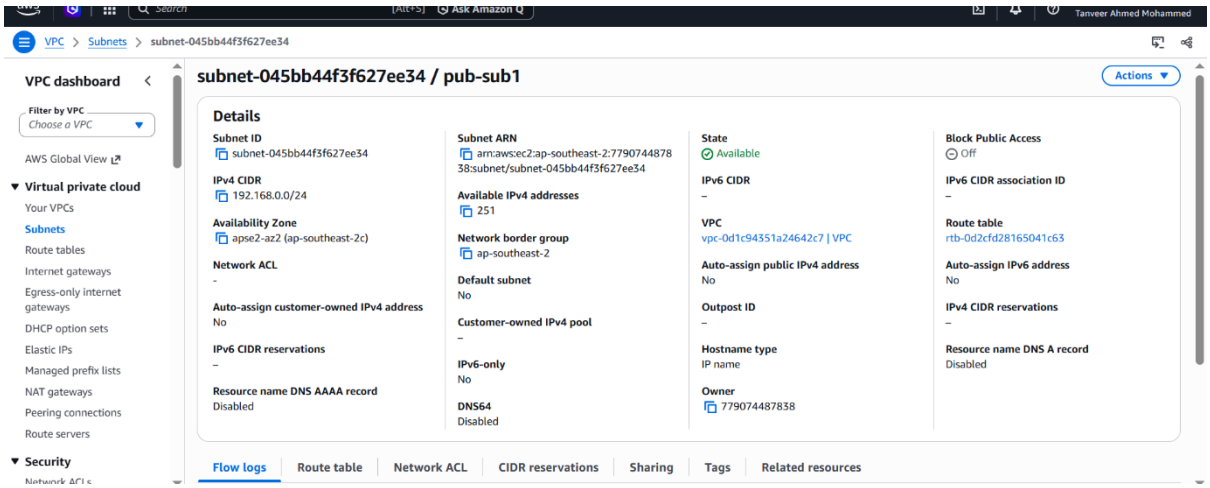


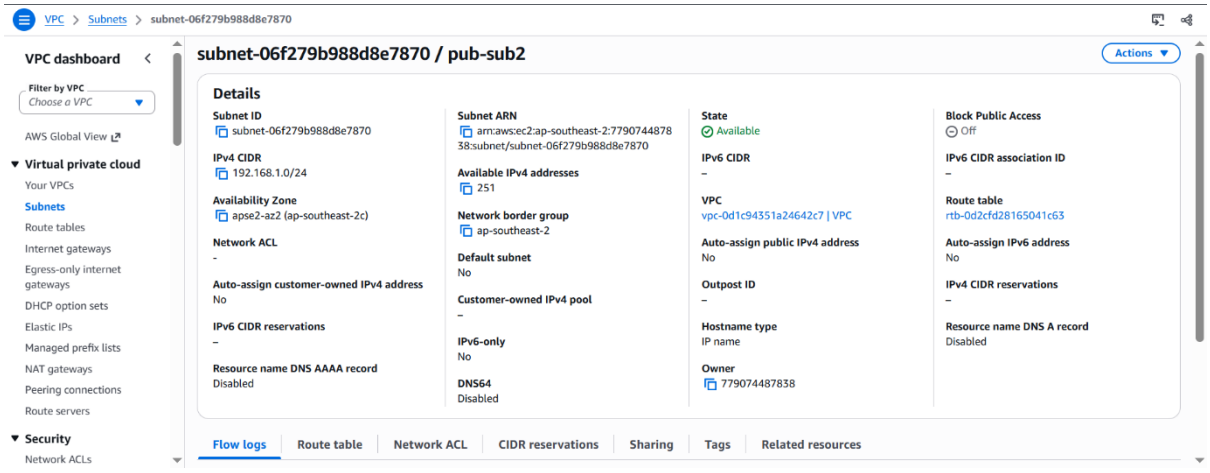


Now Public IP Subnets

- **pub-sub1**
  - **Subnet ID:** subnet-045bb44f3f627ee34
  - **IPv4 CIDR:** 192.168.0.0/24
  - **Availability Zone:** apse2-az2 (ap-southeast-2c)
  - **State:** Available
  - **Block Public Access:** Off
- **pub-sub2**
  - **Subnet ID:** subnet-06f279b988d8e7870
  - **IPv4 CIDR:** 192.168.1.0/24
  - **Availability Zone:** apse2-az2 (ap-southeast-2c)
  - **State:** Available
  - **Block Public Access:** Off







2) Enable DNS Hostname in VPC.

Now we will enable DNS Hostnames and DNS Resolution in your VPC.

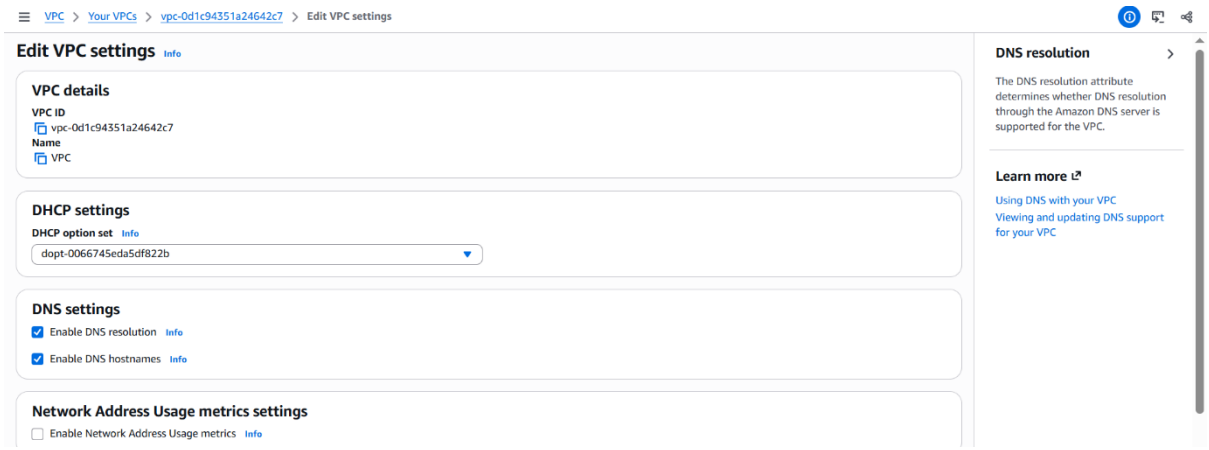
Step 1: Open Your VPCs

1. Sign in to the AWS Console.
2. Search for VPC.
3. Open the VPC service.
4. Click Your VPCs in the left-side menu.
5. Select your VPC: project-vpc.

Step 2: Edit VPC settings

1. Select project-vpc.

2. Click Actions.
3. Select Edit VPC settings.
4. Save



Step 3: Verify

Return to **Your VPCs** and select vpc.

Open the details panel and verify:

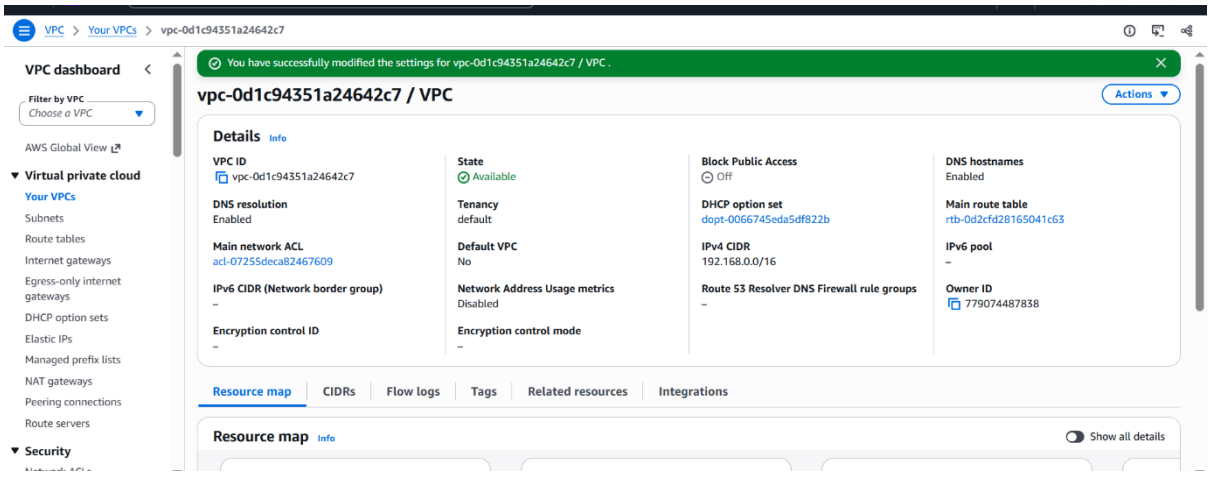
- DNS resolution: Enabled
- DNS hostnames: Enabled

Understand it simply

DNS resolution: Converts domain names into IP addresses.

Example: Resolving google.com to an IP address.

DNS hostnames: Allows supported EC2 instances to receive DNS hostnames.



3) Enable Auto Assign Public IP in 2 public subnets.

4) We will enable automatic public IPv4 address assignment for:
- pub-sub1
- pub-sub2

Step 1: Open Subnets

1. Sign in to the AWS Management Console.

2. Search for VPC.
3. Click VPC.
4. In the left menu, select Subnets.

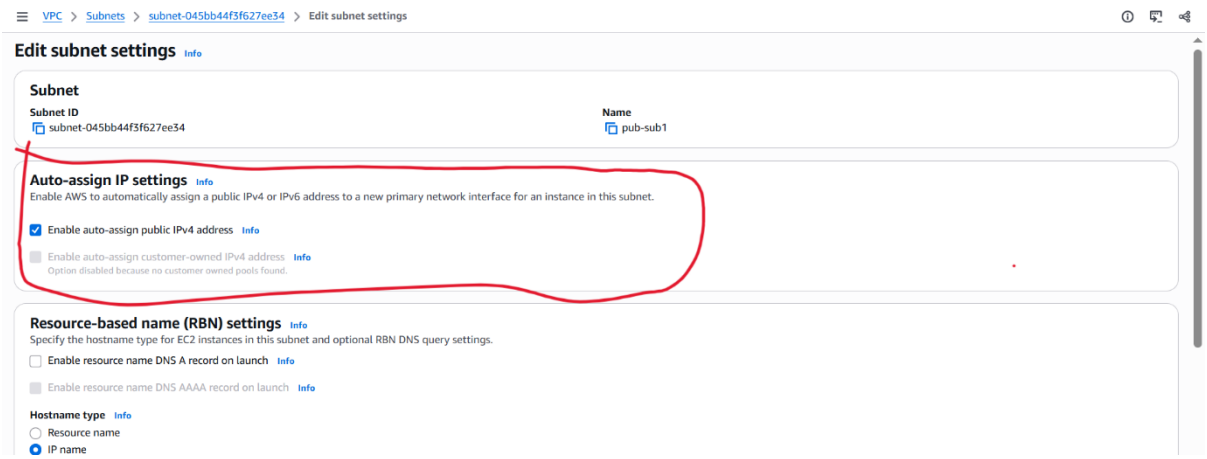
Step 2: Enable it for Public Subnet 1

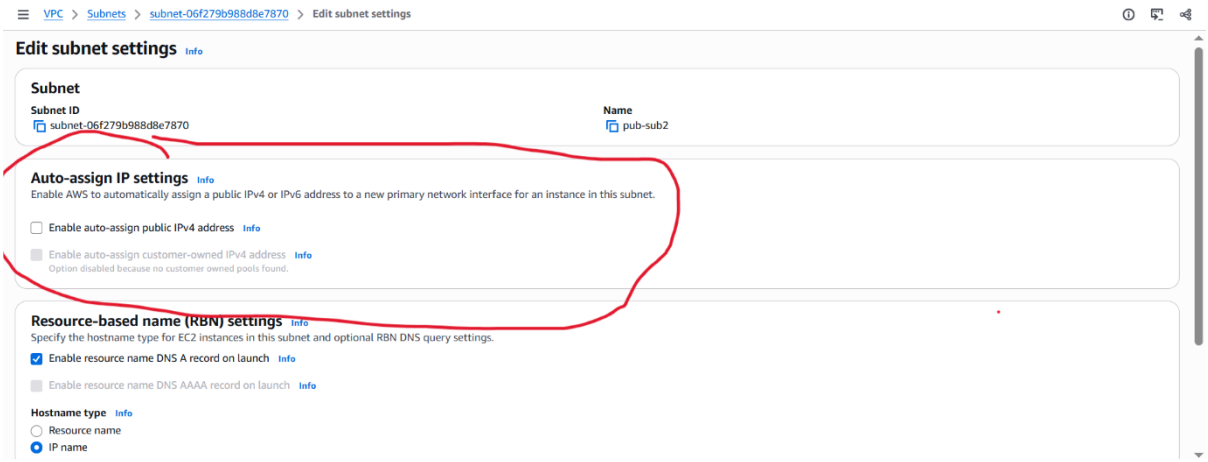
1. Select pub-sub-1.
2. Click Actions.
3. Select Edit subnet settings.
4. Under Auto-assign IP settings, find Enable auto-assign public IPv4 address.
5. ✅ Check the box.
6. Click Save.

Step 3: Enable it for Public Subnet 2

Repeat the same steps:

1. Select pub-subnet-2.
2. Click Actions → Edit subnet settings.
3. Check Enable auto-assign public IPv4 address.
4. Click Save.





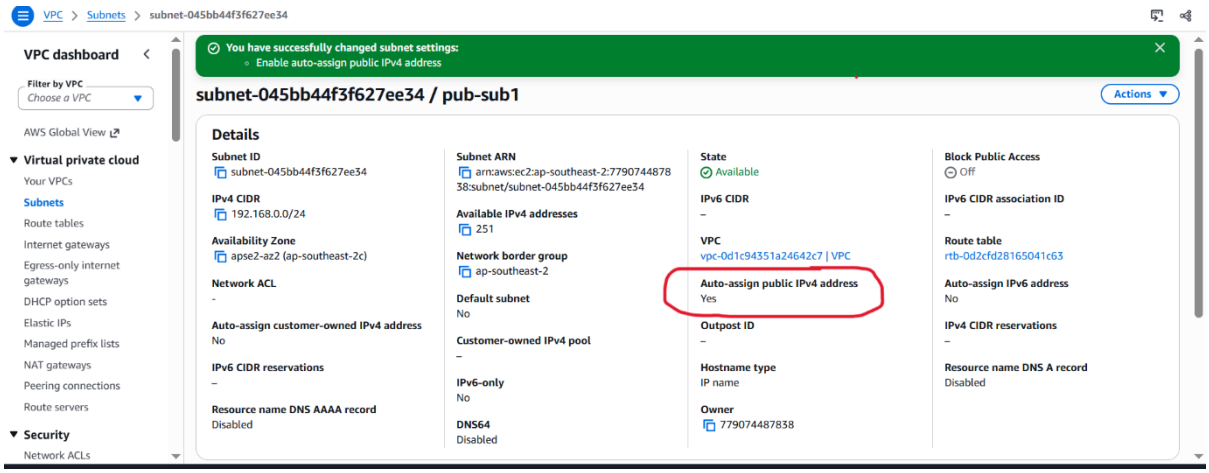
Step 4: Verify both subnets

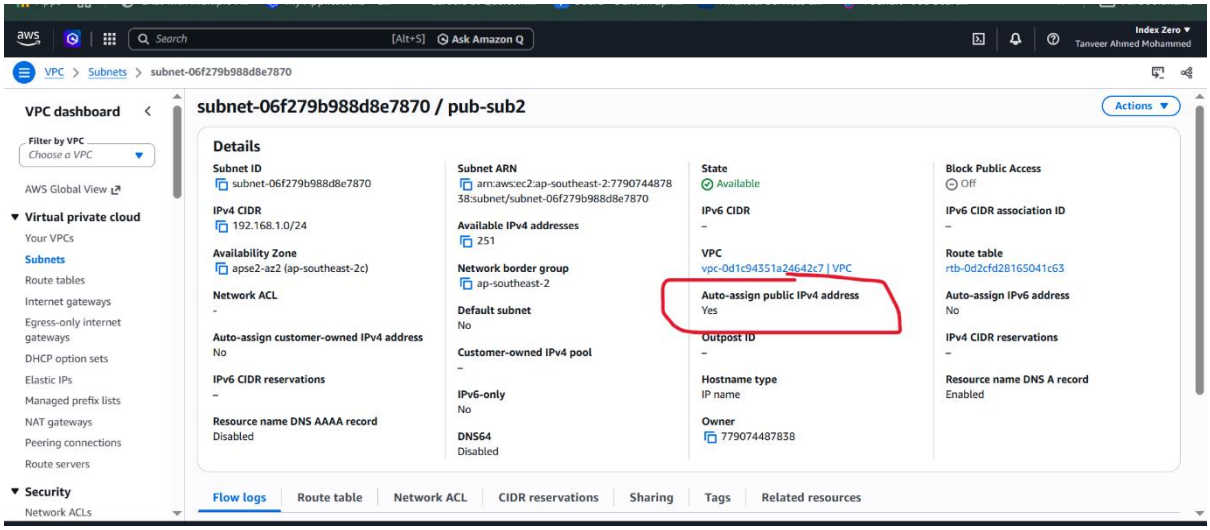
Open each subnet's details and check that Auto-assign public IPv4 address is enabled.

What does this setting do?

When you launch an EC2 instance in a subnet with this setting enabled, AWS can automatically assign it a public IPv4 address if you haven't overridden the setting during launch.

Important: Enabling this setting alone does not make a subnet public. You also need an Internet Gateway and a route to it in the subnet's associated route table.





4)Add 2 private subnets in private route table.

Now we will associate both private subnets with one private route table.

Why? A route table controls where network traffic from a subnet goes. Both private subnets should use the private route table.

Step 1: Open Route Tables

1. Sign in to AWS Console.

2. Search for VPC.

3. Open VPC.

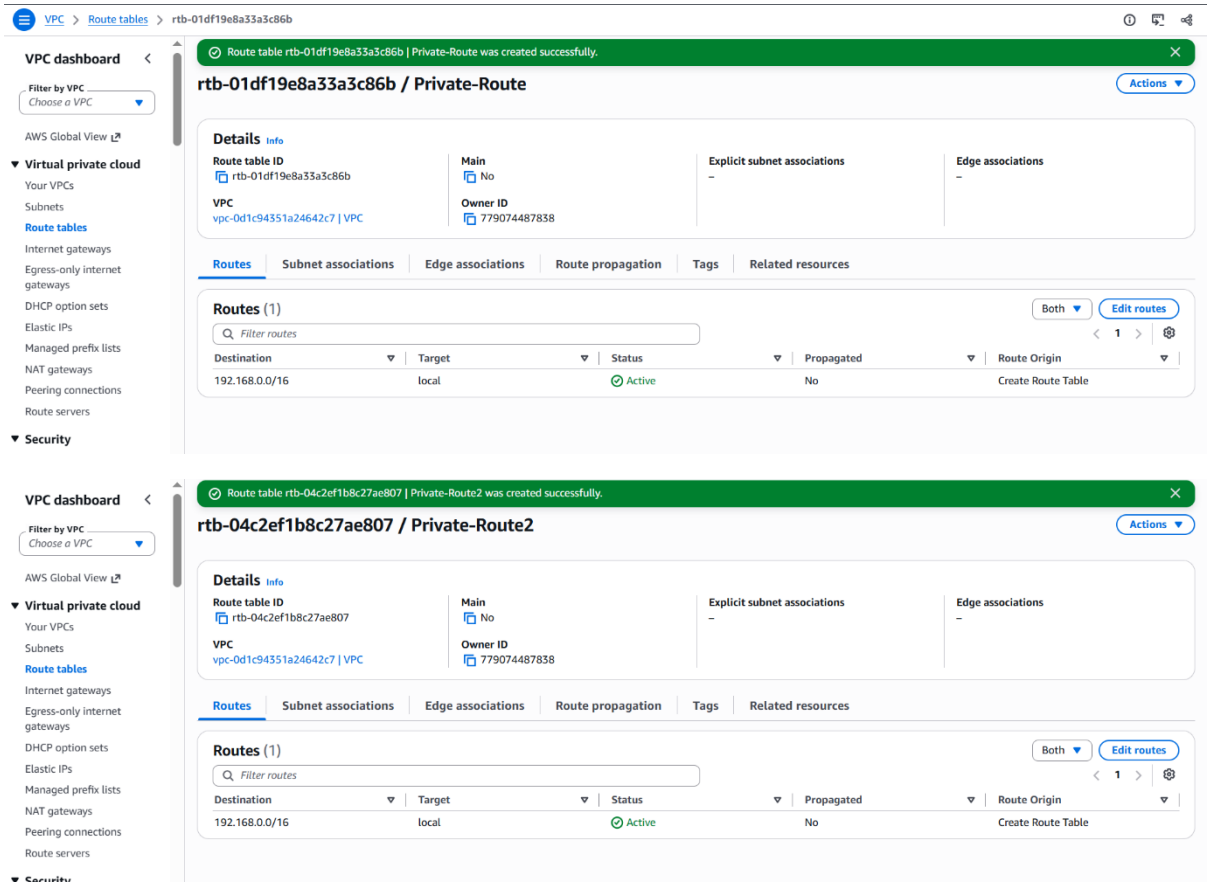
4. In the left-side menu, click Route tables.

Step 2: Create a Private Route Table

If you haven't created one yet:

1. Click Create route table.

2. Enter these details:



Step 3: Associate Private Subnets

1. Select private-rt.

2. Open the Subnet associations tab.

3. Click Edit subnet associations.

4. You will see a list of available subnets.

5. Select these two:

private-subnet-1

CIDR: 10.0.3.0/24

private-subnet-2

CIDR: 10.0.4.0/24

6. Click Save associations.

Step 4: Verify

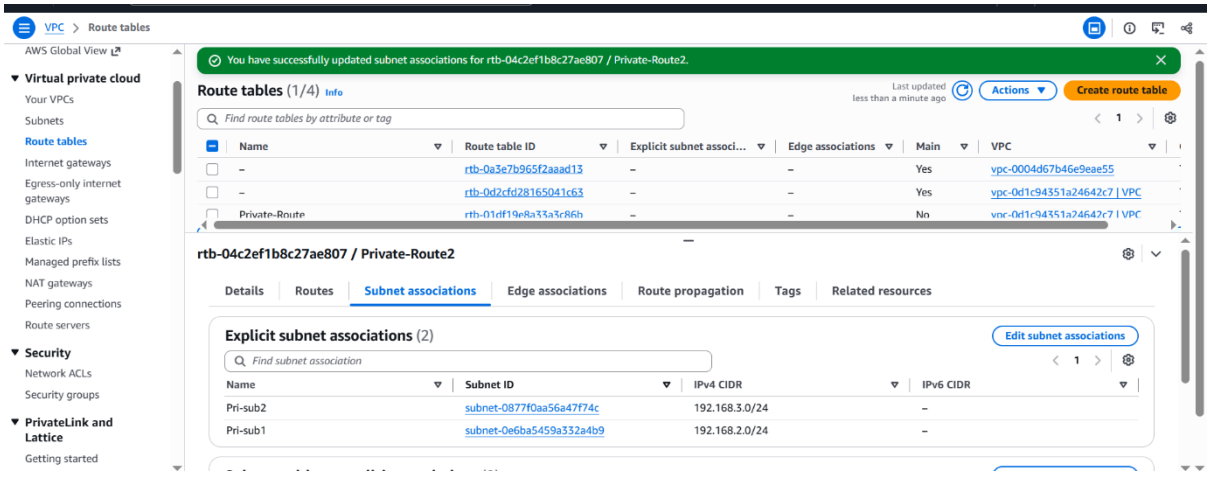
Select private-rt and open Subnet associations.

You should see:

Subnet Association

private-subnet-1 Associated ✅

private-subnet-2 Associated ✅



This local route allows communication between resources inside your VPC.

5)Add 2 public subnets in public route table.

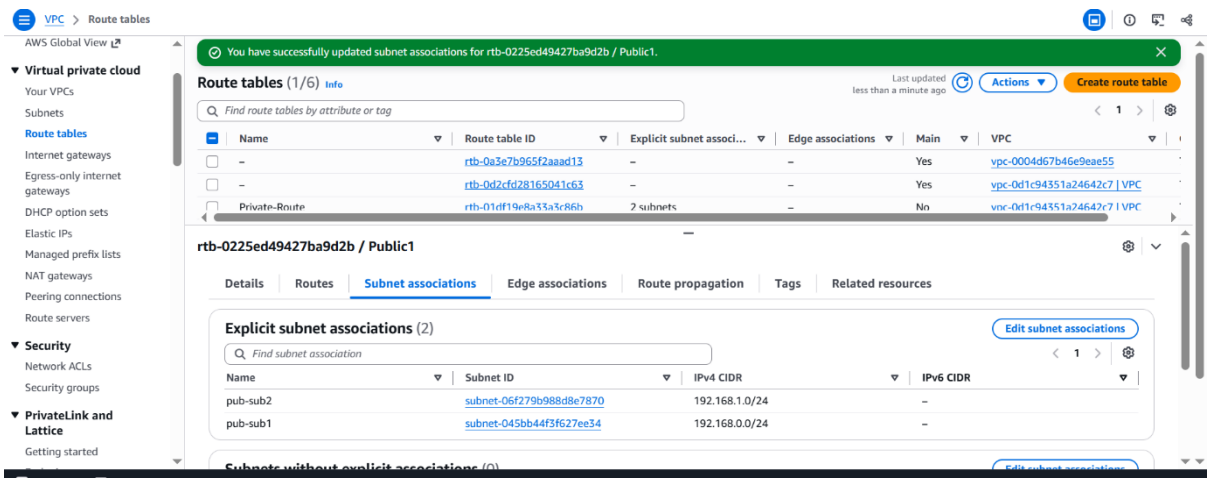
Now we will create a public route table and associate both public subnets with it.

Step 1: Open Route Tables

1. Open AWS Console.
2. Search for VPC.
3. Click Route tables in the left-side menu.

Step 2: Create Public Route Table

1. Click Create route table.



Step 3: Associate Public Subnets

1. Select public-rt.
2. Open the Subnet associations tab.
3. Click Edit subnet associations.
4. Select these two subnets:

- public-subnet-1 — 10.0.1.0/24
- public-subnet-2 — 10.0.2.0/24

5. Click Save associations.

Step 4: Verify

Open the Subnet associations tab.You should see:

Subnet	Association
public-subnet-1	Associated ✅
public-subnet-2	Associated ✅

Step 5: Check Routes

Open the Routes tab.

You should initially see:

Destination	Target
10.0.0.0/16	local

The local route is automatically created by AWS.

6)Public route table will have the routes to internet and local.

Now we will configure the public route table so that resources in public subnets can communicate with the internet.

What is a route?

A route tells AWS where to send network traffic.

We need two routes:

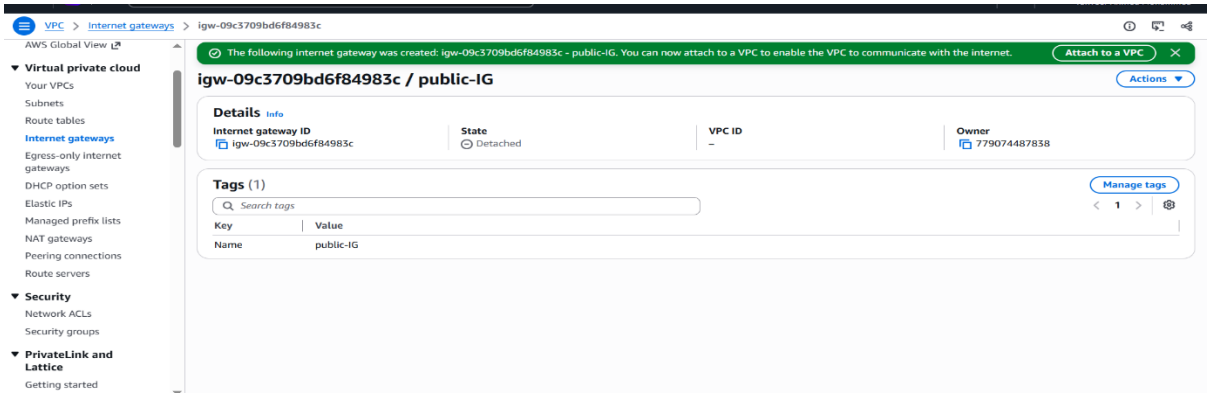
1. Local route – communication inside the VPC.
2. Internet route – communication outside the VPC.

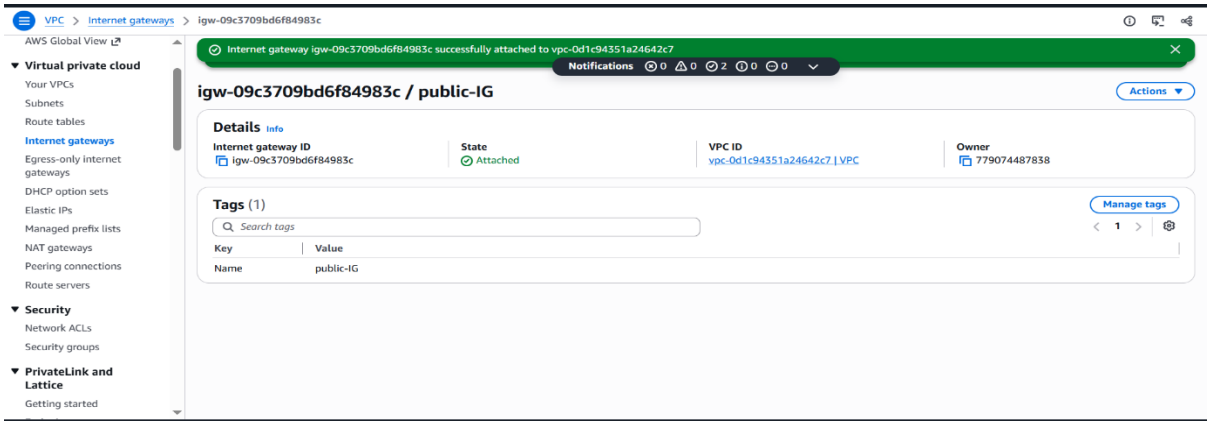
Step 1: Create an Internet Gateway

First, we need an Internet Gateway.

1. Open VPC in AWS Console.
2. In the left menu, select Internet gateways.
3. Click Create internet gateway.
4. Enter:

Setting	Value
Name tag	-igw





Step 2: Attach Internet Gateway to VPC

1. Select project-igw.
2. Click Actions.
3. Select Attach to a VPC.
4. Select project-vpc.
5. Click Attach internet gateway.

Internet Gateway attached

Step 3: Open Public Route Table

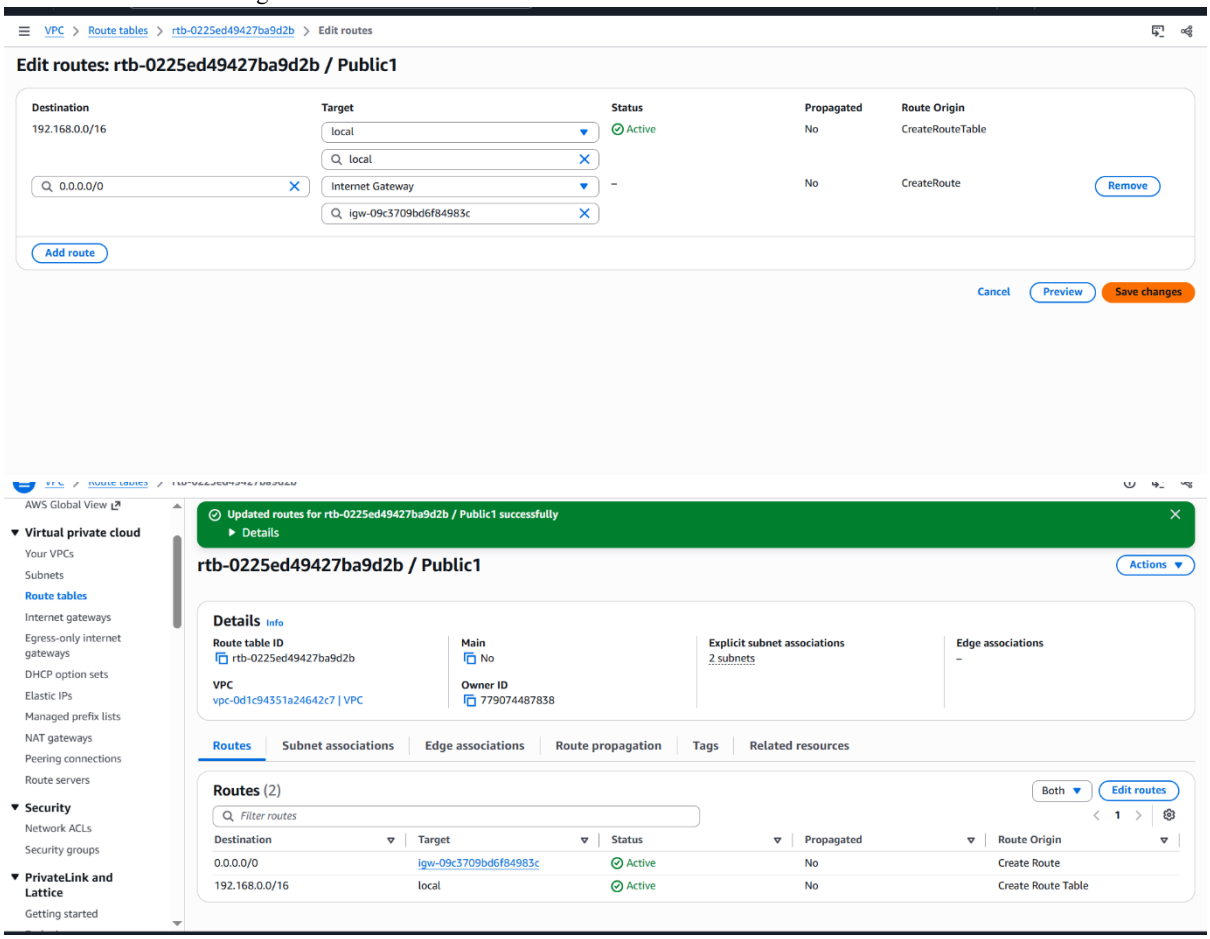
1. Go to VPC → Route tables.
2. Select public-rt.
3. Open the Routes tab.
4. Click Edit routes.

Step 4: Add Internet Route

1. Click Add route.
2. Enter:

Field	Value
Destination	0.0.0.0/0
Target	Internet Gateway
Target value	project-igw

3. Click Save changes.



Step 5: Verify Both Routes

Your public route table should look like this:

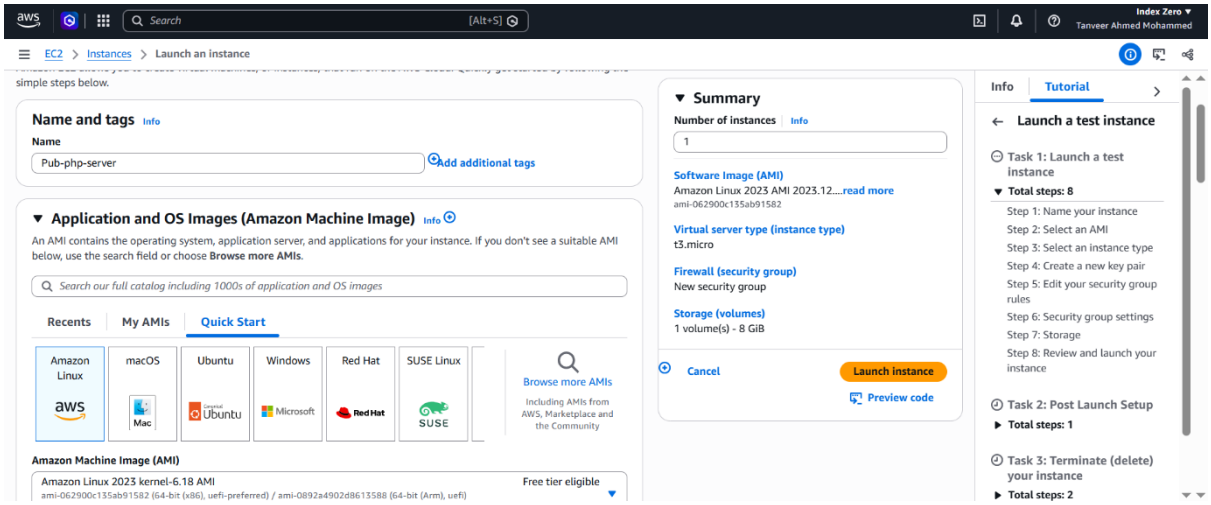
Destination	Target
10.0.0.0/16	local
0.0.0.0/0	igw-xxxxxxxx

The Internet Gateway ID will appear instead of the name.

Understand the routes:
- 10.0.0.0/16 → local: Traffic within your VPC stays inside your VPC.
- 0.0.0.0/0 → Internet Gateway: Traffic destined for other IPv4 networks goes through the Internet Gateway.
Step 6: Final Verification
Check these three things:
- project-igw is attached to project-vpc.
- public-subnet-1 is associated with public-rt.
- public-subnet-2 is associated with public-rt

7) Create EC2 in public subnet with t2. micro and install PHP.

Here I Have Created an EC2 instance named pub-php-server using t2.micro in public-subnet-1.
Step 1: Open EC2
1. Sign in to your AWS Console.
2. Search for EC2.
3. Open the EC2 service.
4. In the left menu, click Instances.
5. Click Launch instances.



Step 2: Configure the instance
Enter the following details:

Setting	Value
Name	pub-php-server
AMI	Amazon Linux 2023
Instance type	t2.micro
Key pair	Create a new key pair or select an existing one
VPC	-vpc
Subnet	public-subnet-1
Auto-assign public IP	Enable

Note: If t2.micro is unavailable in yourStep 3: Configure the security group
Under Network settings, create a new security group.

Security group name:

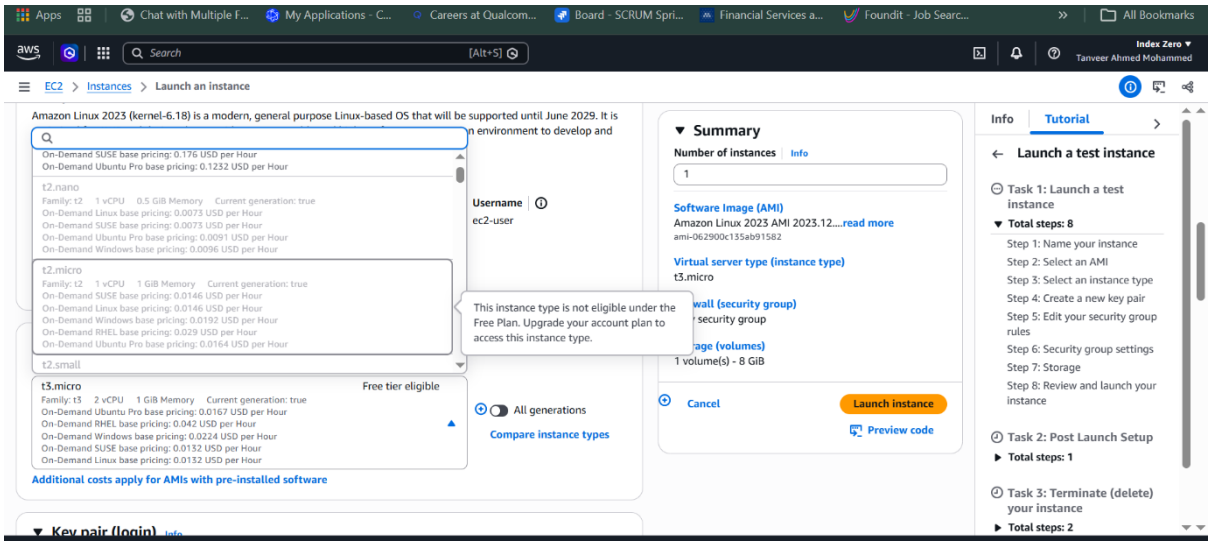
public-php-sg

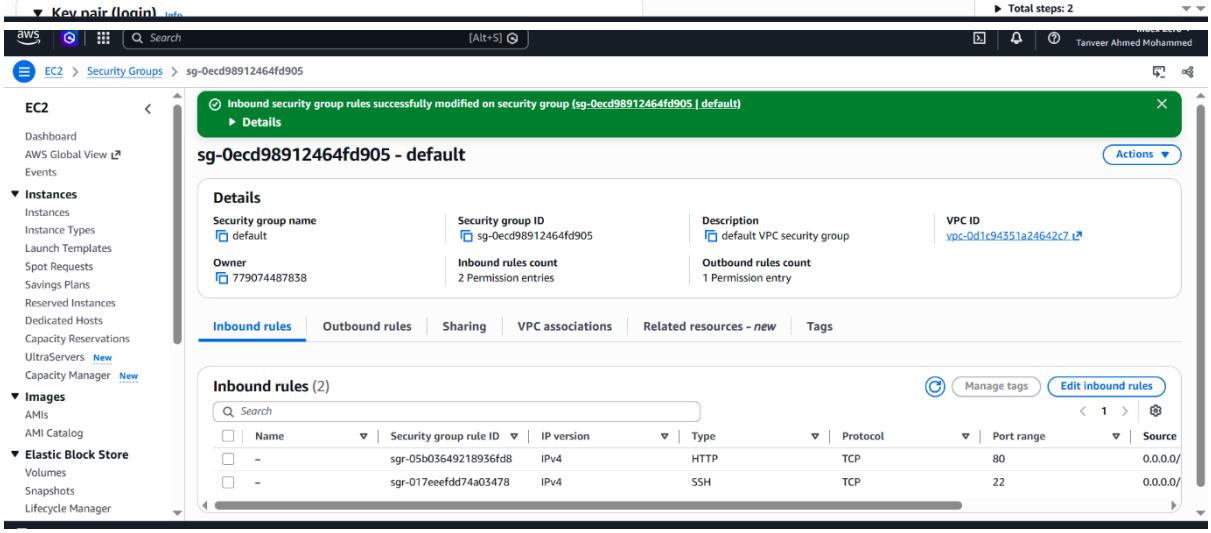
Add these inbound rules:

Type	Port	Source
SSH	22	My IP
HTTP	80	Anywhere IPv4 (0.0.0.0/0) selected Region, check whether another eligible instance type is available.

- **SSH** allows you to connect to the instance.
- **HTTP** allows users to access your website.

Avoid opening SSH to everyone.





Now, here use SSH from your computer with your downloaded key pair.

ssh -i "Linux.pem" ec2-user@ec2-3-107-13-22.ap-southeast-2.compute.amazonaws.com

```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/Downloads
$ ssh -i "Linux.pem" ec2-user@ec2-3-107-13-22.ap-southeast-2.compute.amazonaws.com
The authenticity of host 'ec2-3-107-13-22.ap-southeast-2.compute.amazonaws.com (3.107.13.22)' can't be established.
ED25519 key fingerprint is SHA256:6cmiSWjQYVs6lg4DM5+T1qTwWWQaGbpsXsC8GJJ7LbA.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-3-107-13-22.ap-southeast-2.compute.amazonaws.com' (ED25519) to the list of known hosts.
       #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-192-168-0-250 ~]$ sudo -i
[root@ip-192-168-0-250 ~]# sudo dnf update -y
Amazon Linux 2023 Kernel Livepatch repository                                                    791 kB/s |  89 kB
Dependencies resolved.
Nothing to do.
Complete!
[root@ip-192-168-0-250 ~]# sudo dnf install -y httpd php
Last metadata expiration check: 0:00:14 ago on Sat Oct 31 10:07:33 2026.
Dependencies resolved.
================================================================================================================================
 Package                                Architecture            Version                                 Repository
================================================================================================================================
Installing:
 httpd                                  x86_64                  2.4.69-1.amzn2023.0.2                   amazonlinux
 php8.5                                 x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
Installing dependencies:
 apr                                    x86_64                  1.7.5-1.amzn2023.0.4                    amazonlinux
 apr-util                               x86_64                  1.6.5-1.amzn2023.0.1                    amazonlinux
 apr-util-lmdb                          x86_64                  1.6.5-1.amzn2023.0.1                    amazonlinux
 generic-logos-httpd                    noarch                  18.0.0-12.amzn2023.0.3                  amazonlinux
 httpd-core                             x86_64                  2.4.69-1.amzn2023.0.2                   amazonlinux
 httpd-filesystem                       noarch                  2.4.69-1.amzn2023.0.2                   amazonlinux
 httpd-tools                            x86_64                  2.4.69-1.amzn2023.0.2                   amazonlinux
 libbrotli                              x86_64                  1.1.0-1.amzn2023.0.2                    amazonlinux
 libsodium                              x86_64                  1.0.19-5.amzn2023                       amazonlinux
 mailcap                                noarch                  2.1.49-3.amzn2023.0.3                   amazonlinux
```

Step 6: Install Apache and PHP

Run these commands in the EC2 terminal, one at a time.

After Switching ec2 to root User.

Sudo -i

Update packages: sudo dnf update -y

Install Apache and PHP:

sudo dnf install -y httpd php

Start Apache:

sudo systemctl enable --now httpd

Check Apache status:

sudo systemctl status httpd

```
 libsodium                              x86_64                  1.0.19-5.amzn2023                       amazonlinux
 mailcap                                noarch                  2.1.49-3.amzn2023.0.3                   amazonlinux
 nginx-filesystem                       noarch                  1:1.30.6-1.amzn2023.0.1                 amazonlinux
 php8.5-cli                             x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-common                          x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-process                         x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-xml                             x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
Installing weak dependencies:
 apr-util-openssl                       x86_64                  1.6.5-1.amzn2023.0.1                    amazonlinux
 mod_http2                              x86_64                  1:2.4.69-1.amzn2023.0.2                 amazonlinux
 mod_lua                                x86_64                  2.4.69-1.amzn2023.0.2                   amazonlinux
 php8.5-fpm                             x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-mbstring                        x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-pdo                             x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux
 php8.5-sodium                          x86_64                  8.5.12-1.amzn2023.0.1                   amazonlinux

Transaction Summary
================================================================================================================================
Install  24 Packages

Total download size: 14 M
Installed size: 66 M
Downloading Packages:
(1/24): apr-util-1.6.5-1.amzn2023.0.1.x86_64.rpm                                 4.5 MB/s | 100 kB
(2/24): apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64.rpm                            582 kB/s |  15 kB
(3/24): apr-1.7.5-1.amzn2023.0.4.x86_64.rpm                                      4.0 MB/s | 129 kB
(4/24): apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64.rpm                         697 kB/s |  17 kB
(5/24): generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch.rpm                    712 kB/s |  19 kB
(6/24): httpd-2.4.69-1.amzn2023.0.2.x86_64.rpm                                   1.8 MB/s |  45 kB
(7/24): httpd-core-2.4.69-1.amzn2023.0.2.x86_64.rpm                               36 MB/s | 1.4 MB
(8/24): httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch.rpm                        332 kB/s |  12 kB
(9/24): httpd-tools-2.4.69-1.amzn2023.0.2.x86_64.rpm                             2.2 MB/s |  79 kB
(10/24): libbrotli-1.1.0-1.amzn2023.0.2.x86_64.rpm                                11 MB/s | 333 kB
(11/24): mailcap-2.1.49-3.amzn2023.0.3.noarch.rpm                                1.2 MB/s |  33 kB
(12/24): mod_http2-2.4.69-1.amzn2023.0.2.x86_64.rpm                              5.3 MB/s | 130 kB
(13/24): mod_lua-2.4.69-1.amzn2023.0.2.x86_64.rpm                                2.3 MB/s |  58 kB
(14/24): libsodium-1.0.19-5.amzn2023.x86_64.rpm                                  2.8 MB/s | 174 kB
(15/24): nginx-filesystem-1.30.6-1.amzn2023.0.1.noarch.rpm                       415 kB/s |  10 kB
(16/24): php8.5-8.5.12-1.amzn2023.0.1.x86_64.rpm                                 487 kB/s |  13 kB
(17/24): php8.5-cli-8.5.12-1.amzn2023.0.1.x86_64.rpm                              55 MB/s | 6.2 MB
(18/24): php8.5-common-8.5.12-1.amzn2023.0.1.x86_64.rpm                          8.0 MB/s | 870 kB
(19/24): php8.5-fpm-8.5.12-1.amzn2023.0.1.x86_64.rpm                              26 MB/s | 3.1 MB
(20/24): php8.5-pdo-8.5.12-1.amzn2023.0.1.x86_64.rpm                             1.3 MB/s | 100 kB
(21/24): php8.5-mbstring-8.5.12-1.amzn2023.0.1.x86_64.rpm                        5.5 MB/s | 538 kB
```

```
  Verifying        : php8.5-mbstring-8.5.12-1.amzn2023.0.1.x86_64
  Verifying        : php8.5-pdo-8.5.12-1.amzn2023.0.1.x86_64
  Verifying        : php8.5-process-8.5.12-1.amzn2023.0.1.x86_64
  Verifying        : php8.5-sodium-8.5.12-1.amzn2023.0.1.x86_64
  Verifying        : php8.5-xml-8.5.12-1.amzn2023.0.1.x86_64

Installed:
  apr-1.7.5-1.amzn2023.0.4.x86_64                apr-util-1.6.5-1.amzn2023.0.1.x86_64                apr-util-lmdb-1.6.5-1.amzn2023.0.1.x86_64
  apr-util-openssl-1.6.5-1.amzn2023.0.1.x86_64   generic-logos-httpd-18.0.0-12.amzn2023.0.3.noarch   httpd-2.4.69-1.amzn2023.0.2.x86_64
  httpd-core-2.4.69-1.amzn2023.0.2.x86_64        httpd-filesystem-2.4.69-1.amzn2023.0.2.noarch       httpd-tools-2.4.69-1.amzn2023.0.2.x86_64
  libbrotli-1.1.0-1.amzn2023.0.2.x86_64          libsodium-1.0.19-5.amzn2023.x86_64                  mailcap-2.1.49-3.amzn2023.0.3.noarch
  mod_http2-1:2.4.69-1.amzn2023.0.2.x86_64       mod_lua-2.4.69-1.amzn2023.0.2.x86_64                nginx-filesystem-1:1.30.6-1.amzn2023.0.1.n
  php8.5-8.5.12-1.amzn2023.0.1.x86_64            php8.5-cli-8.5.12-1.amzn2023.0.1.x86_64             php8.5-common-8.5.12-1.amzn2023.0.1.x86_64
  php8.5-fpm-8.5.12-1.amzn2023.0.1.x86_64        php8.5-mbstring-8.5.12-1.amzn2023.0.1.x86_64        php8.5-pdo-8.5.12-1.amzn2023.0.1.x86_64
  php8.5-process-8.5.12-1.amzn2023.0.1.x86_64    php8.5-sodium-8.5.12-1.amzn2023.0.1.x86_64          php8.5-xml-8.5.12-1.amzn2023.0.1.x86_64

Complete!
[root@ip-192-168-0-250 ~]# sudo systemctl enable --now httpd
Created symlink /etc/systemd/system/multi-user.target.wants/httpd.service → /usr/lib/systemd/system/httpd.service.
[root@ip-192-168-0-250 ~]# sudo systemctl status httpd
● httpd.service - The Apache HTTP Server
     Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
    Drop-In: /usr/lib/systemd/system/httpd.service.d
             └─php-fpm.conf
     Active: active (running) since Sat 2026-10-31 10:08:13 UTC; 16s ago
       Docs: man:httpd.service(8)
   Main PID: 26464 (httpd)
     Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/sec:   0 B/sec"
      Tasks: 177 (limit: 1059)
     Memory: 13.9M
        CPU: 89ms
     CGroup: /system.slice/httpd.service
             ├─26464 /usr/sbin/httpd -DFOREGROUND
             ├─26466 /usr/sbin/httpd -DFOREGROUND
             ├─26467 /usr/sbin/httpd -DFOREGROUND
             ├─26468 /usr/sbin/httpd -DFOREGROUND
             └─26514 /usr/sbin/httpd -DFOREGROUND

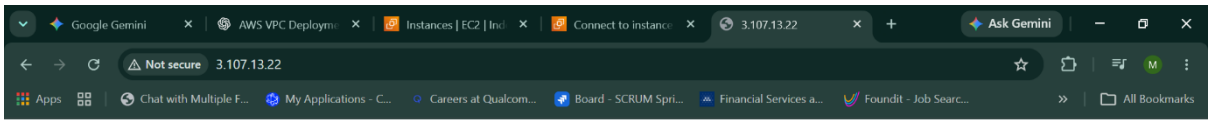
Oct 31 10:08:13 ip-192-168-0-250.ap-southeast-2.compute.internal systemd[1]: Starting httpd.service - The Apache HTTP Server...
Oct 31 10:08:13 ip-192-168-0-250.ap-southeast-2.compute.internal systemd[1]: Started httpd.service - The Apache HTTP Server.
Oct 31 10:08:14 ip-192-168-0-250.ap-southeast-2.compute.internal httpd[26464]: Server configured, listening on: port 80
[root@ip-192-168-0-250 ~]# echo '<?php echo "Hello from PHP on AWS EC2!"; ?>' | sudo tee /var/www/html/index.php
<?php echo "Hello from PHP on AWS EC2!"; ?>
[root@ip-192-168-0-250 ~]#
```

Step 7: Create a PHP test page

echo '<?php echo "Hello from PHP on AWS EC2!"; ?>' | sudo tee /var/www/html/index.php

Step 8: Test your website

1. Go to EC2 → Instances.
2. Select pub-php-server.
3. Copy its Public IPv4 address.
4. Open your browser.



Hello from PHP on AWS EC2!

I launched an EC2 instance in a public subnet, configured its security group, installed Apache and PHP, deployed a PHP test page, and verified website accessibility through the EC2 public IP.

8)Configure NAT gateway in public subnet and connect to private instance.

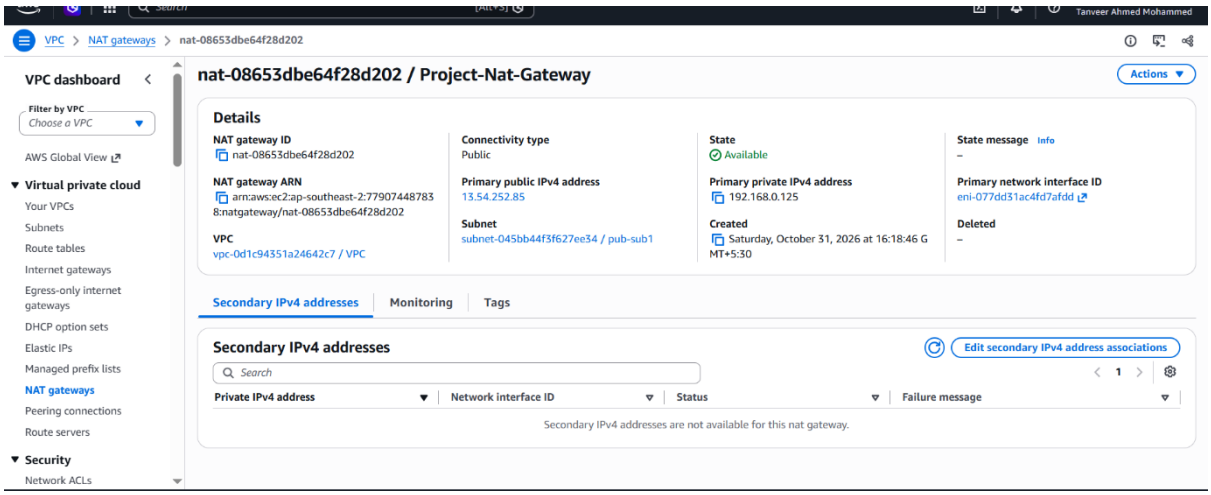
Part 1: Configure NAT Gateway in the public subnet

1. Open AWS Console → VPC → NAT Gateways.
2. Click Create NAT Gateway and enter these details:
   - Name: project-nat
   - Subnet: public-subnet-1
   - Connectivity type: Public
   - Elastic IP: Allocate Elastic IP
3. Click Create NAT Gateway and wait until the status becomes Available.
4. Go to VPC → Route Tables → private-rt → Routes → Edit routes.
5. Add this route:

Destination	Target
0.0.0.0/0	project-nat

6. Keep the existing local route 10.0.0.0/16. Save your changes.

This allows private EC2 instances to access the internet through the NAT Gateway without having public IP addresses.



Part 2: Launch a private EC2 instance

1. Open EC2 → Launch instance.
2. Configure the instance:

Setting	Value
Name	private-tomcat-server
AMI	Amazon Linux 2023
Instance type	t2.micro
VPC	project-vpc
Subnet	private-subnet-1
Auto-assign public IP	Disable
Security group	private-tomcat-sg

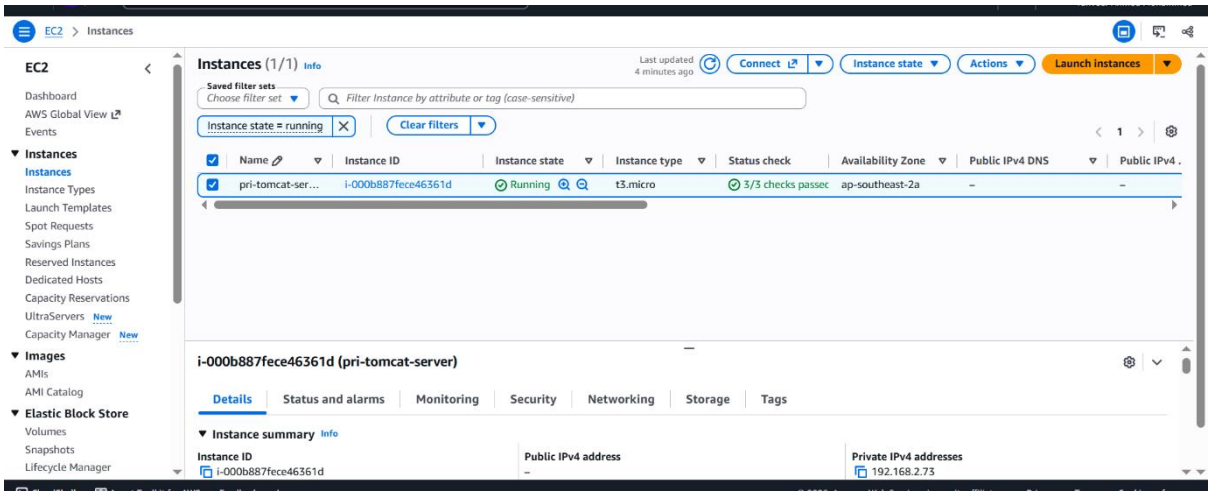
3. In the security group, add these inbound rules:

Type	Port	Source
SSH	22	public-php-sg

Custom TCP 8080 public-php-sg

Here, public-php-sg is the security group attached to your public EC2. Using the security group as the source restricts access to instances using that group.

4. Launch the instance and wait until it is running.



Part 3: Connect to the private EC2 from the public EC2

Your public EC2 will act as a Bastion Host (Jump Server).

Your computer

Public EC2 — public-php-server

Public IP • Bastion Host

Private EC2 — private-tomcat-server

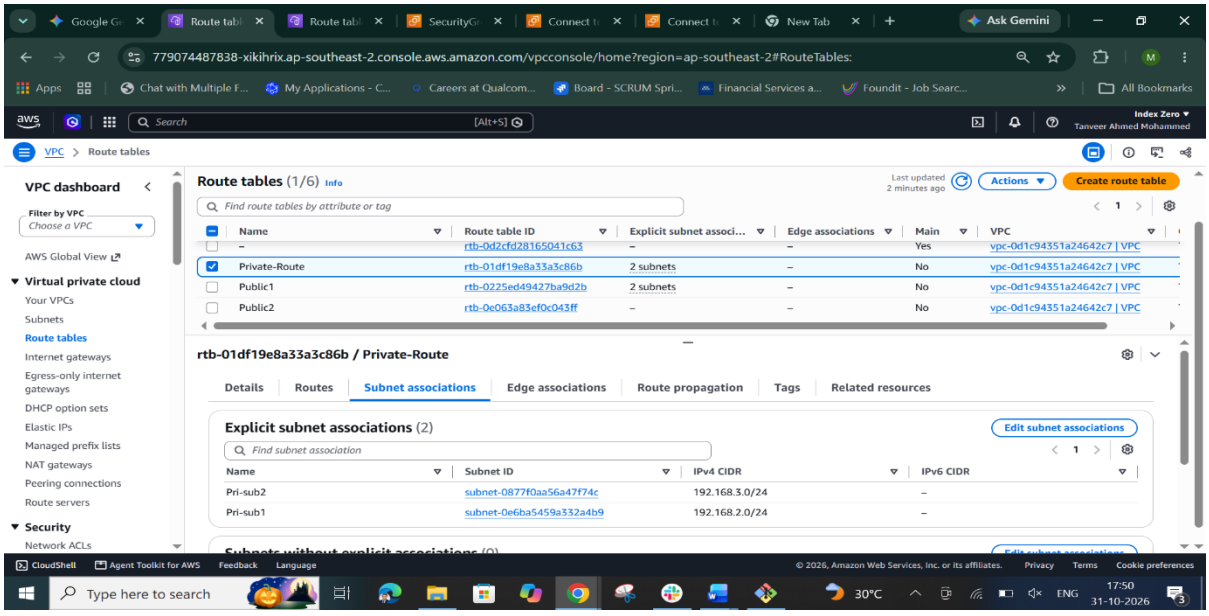
1 First, connect to your public EC2 using SSH or EC2 Instance Connect.

2 From the public EC2, connect to the private EC2 using its Private IPv4 address:

ssh -i "Linux.pem" ec2-user@192.168.2.73

This command assumes the private key is securely available on the public EC2. Do not copy your private key onto the public server casually; a safer approach is SSH agent forwarding or AWS Systems Manager Session Manager.

Step 1: Created and Verified the NAT GatewayAllocated an Elastic IP and created the NAT Gateway (Project-Nat-Gateway) in your public subnet, ensuring it reached the Available state with the IP 13.54.252.85. Updated your private route table (Private-Route) to route all outbound internet traffic (0.0.0.0/0) through your new NAT Gateway. Step 2: Diagnosed and Resolved SSH Connection Hurdles to the Private InstanceIdentified why connecting from the public bastion host (192.168.0.250) to the private instance (192.168.2.73) returned a Permission denied error.Handled the missing .pem key issue on the bastion host by ensuring the key file is created in the correct user directory (ec2-user), securing its file permissions (chmod 400), and establishing the connection path.




```
mdtan@DESKTOP-T7V22P3 MINGW64 ~/Downloads
comsh -i "Linux.pem" ec2-user@ec2-54-206-84-58.ap-southeast-2.compute.amazonaws.c
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
Last login: Sat Oct 31 11:56:20 2026 from 103.203.172.230
[ec2-user@ip-192-168-0-250 ~]$ ssh -i "Linux.pem" ec2-user@192.168.2.73
Warning: Identity file Linux.pem not accessible: No such file or directory.
ec2-user@192.168.2.73: Permission denied (publickey,gssapi-keyex,gssapi-with-mic).
[ec2-user@ip-192-168-0-250 ~]$ eval "$(ssh-agent -s)"
ssh-add Linux.pem
Agent pid 5856
```

```
[ec2-user@ip-192-168-0-250 ~]$ pwd
/home/ec2-user
[ec2-user@ip-192-168-0-250 ~]$ nano Linux.pem
[ec2-user@ip-192-168-0-250 ~]$ chmod 400 Linux.pem
ssh -i Linux.pem ec2-user@192.168.2.73
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \_#####\
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023
   ~~       V~' '->
    ~~~         /
      ~~._.   _/
         _/ _/
       _/m/'
[ec2-user@ip-192-168-2-73 ~]$
```

9) Install Apache Tomcat in private EC2 and deploy a sample app.

Update Packages and Install Java 21

Apache Tomcat 11 requires Java 21 or later to run.

Where to run: Private EC2 terminal prompt.

Execute the commands:

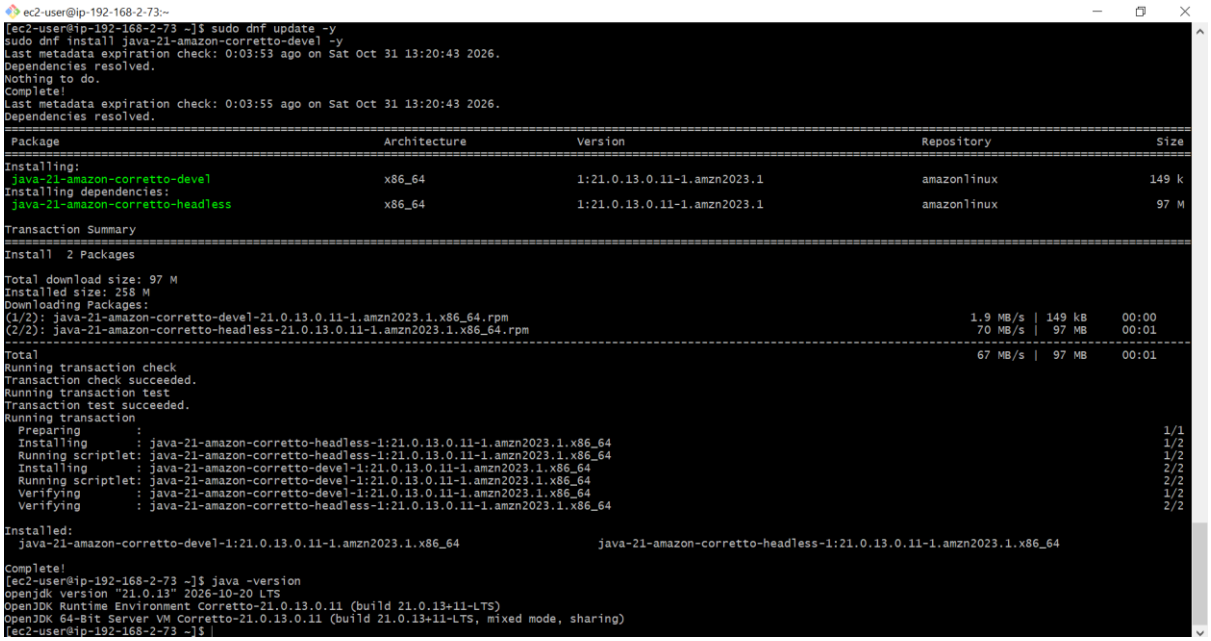
sudo dnf update -y

sudo dnf install java-21-amazon-corretto-devel -y

Verify the installation:

Bash

java -version



Step 2: Create a Dedicated Tomcat System User

For security best practices, run Tomcat under an unprivileged, non-login system user rather than root.

Where to run: Private EC2 terminal prompt.

Execute the commands:

sudo groupadd tomcat

sudo useradd -M -s /bin/nologin -g tomcat -d /opt/tomcat tomcat

The user and group are created successfully without error messages.

Step 3: Download and Extract Apache Tomcat 11

Download the latest binary package for Apache Tomcat 11 and extract it into /opt/tomcat.

VERSION=11.0.27

cd /tmp

wget https://archive.apache.org/dist/tomcat/tomcat-11/v${VERSION}/bin/apache-tomcat-${VERSION}.tar.gz

sudo mkdir -p /opt/tomcat

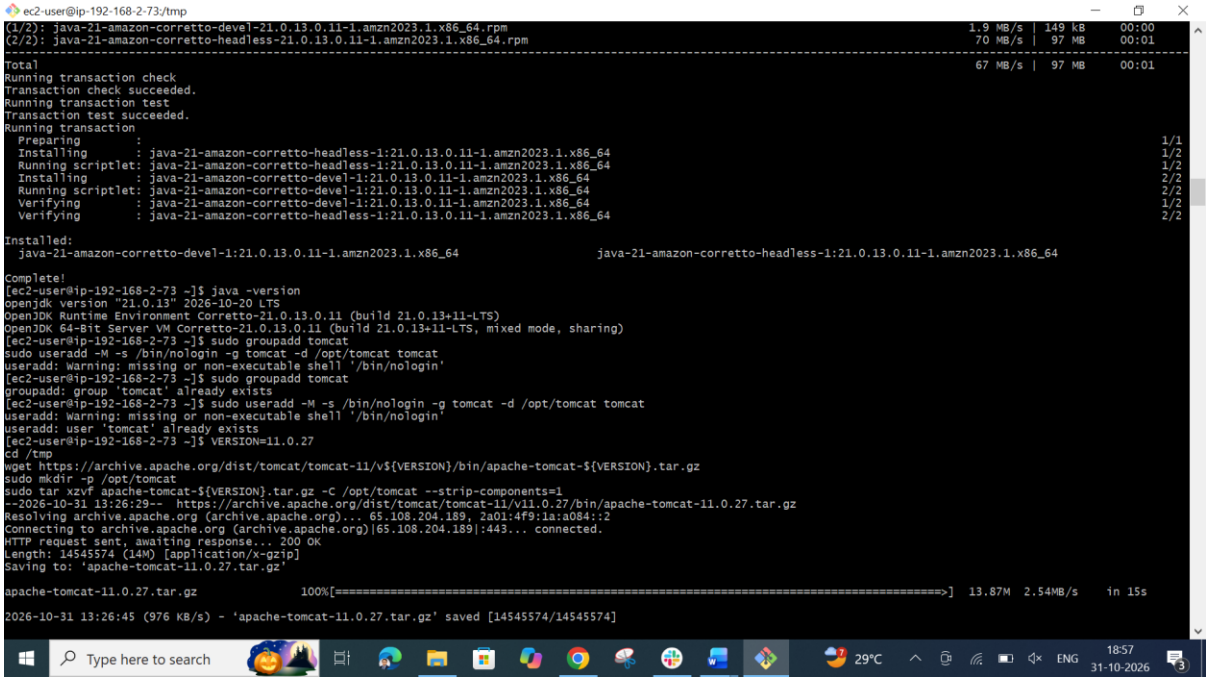
sudo tar xzvf apache-tomcat-${VERSION}.tar.gz -C /opt/tomcat --strip-components=1

Verify extraction:

Bash

ls /opt/tomcat

You should see standard directories such as bin, conf, lib, logs, and webapps.



Step 5: Create a Systemd Service File
Configuring systemd allows Tomcat to run in the background and automatically start up if the instance reboots.

Where to run: Private EC2 terminal prompt.

Open the nano editor to create the service file:Paste the script

Bash
sudo nano /etc/systemd/system/tomcat.service
[Unit]
Description=Apache Tomcat 11 Web Application Container
After=network.target

[Service]
Type=forking
User=tomcat
Group=tomcat
Environment="JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto"
Environment="CATALINA_PID=/opt/tomcat/temp/tomcat.pid"
Environment="CATALINA_HOME=/opt/tomcat"
Environment="CATALINA_BASE=/opt/tomcat"
ExecStart=/opt/tomcat/bin/startup.sh
ExecStop=/opt/tomcat/bin/shutdown.sh
RestartSec=10
Restart=always

[Install]
WantedBy=multi-user.target

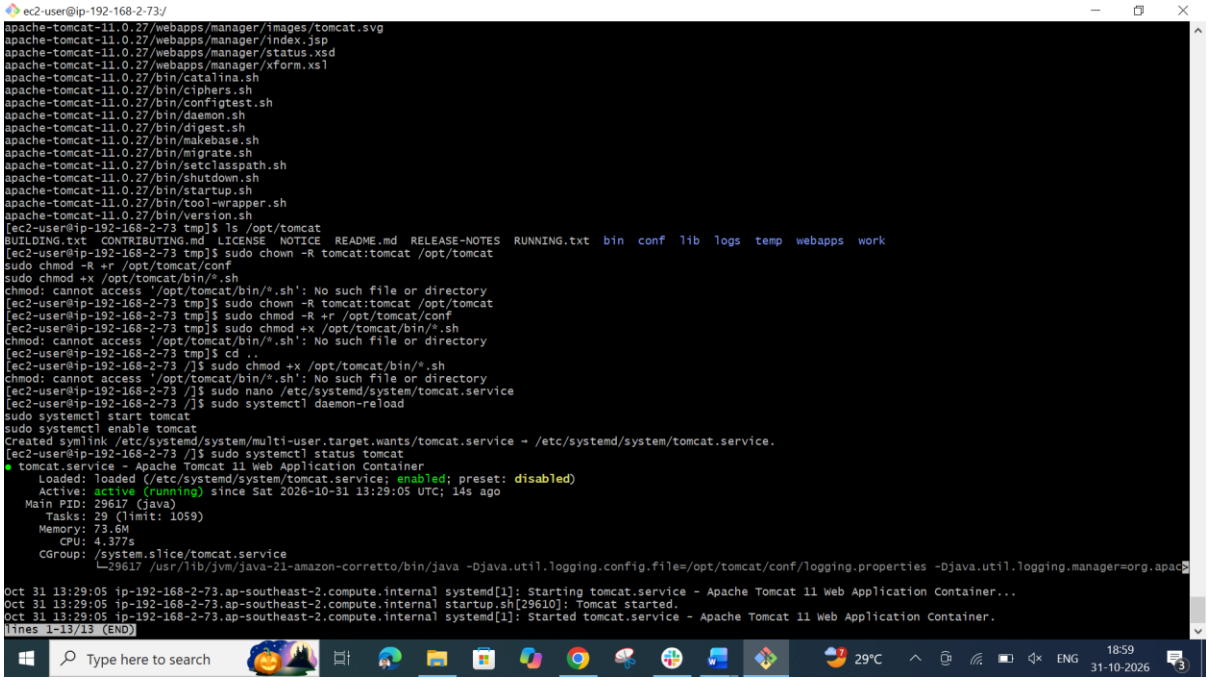
Save and Exit:

- Press Ctrl + O, then press Enter to save.
- Press Ctrl + X to exit the nano editor.

Step 6: Start and Enable the Tomcat Service

1. **Where to run:** Private EC2 terminal prompt.
2. **Execute the commands:**
   sudo systemctl daemon-reload
   sudo systemctl start tomcat
   sudo systemctl enable tomcat
   Check the status:

   Bash
   sudo systemctl status tomcat
   Verification: The service status must show active (running) in green text.



Step 7: Deploy a Sample Application

Download an official sample web application package (.war) directly into the Tomcat webapps directory.

Where to run: Private EC2 terminal prompt.

sudo wget https://tomcat.apache.org/tomcat-11.0-doc/appdev/sample/sample.war -P /opt/tomcat/webapps/

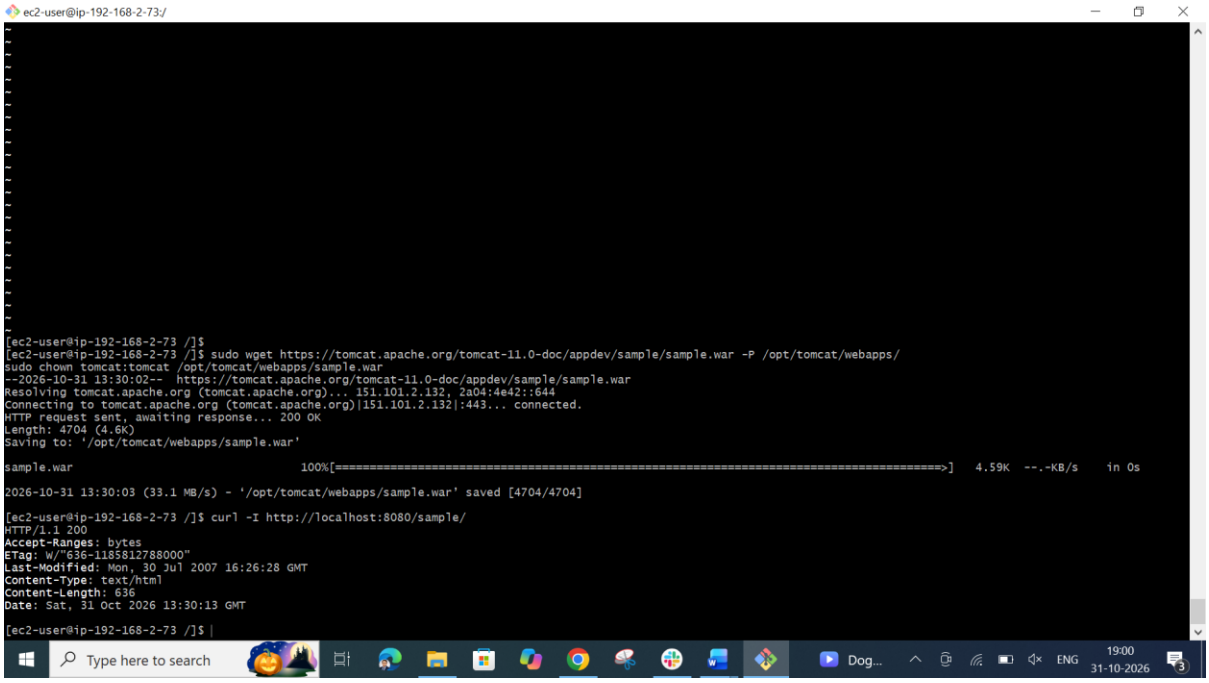
sudo chown tomcat:tomcat /opt/tomcat/webapps/sample.war

Here: Tomcat automatically detects any .war file placed inside the webapps folder and deploys it immediately.

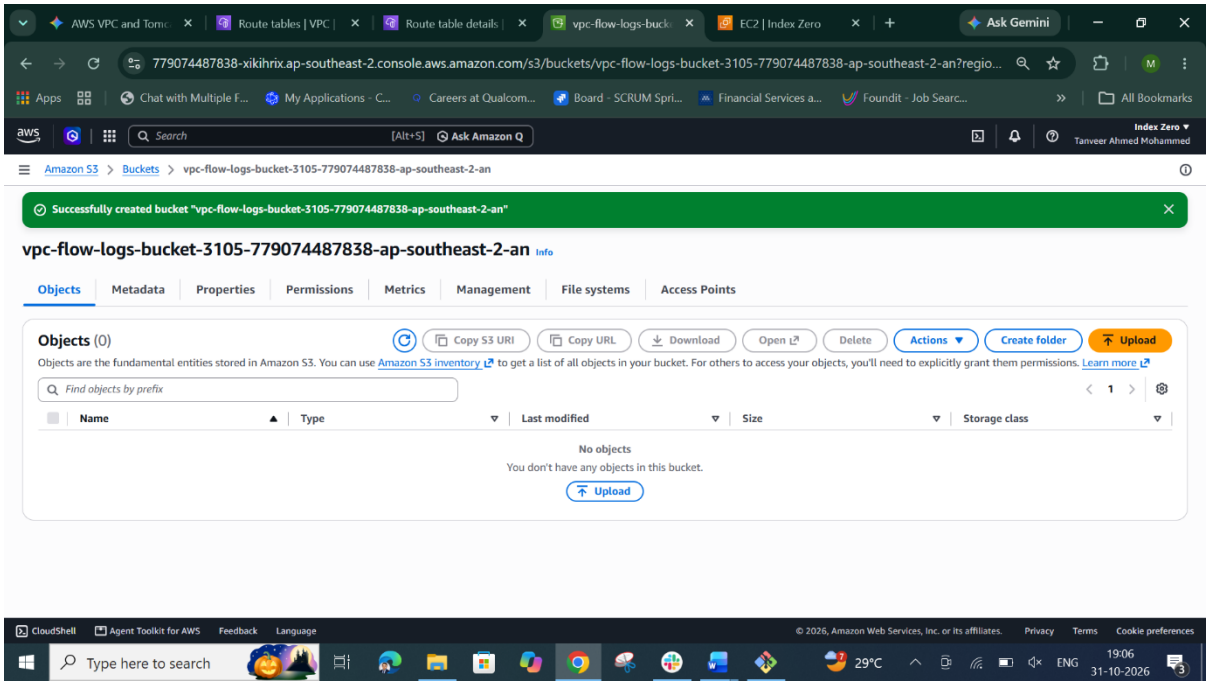
Verify Local Deployment: Run a local curl command inside the private instance to ensure Tomcat is successfully serving the sample app:

Bash

curl -I http://localhost:8080/sample/

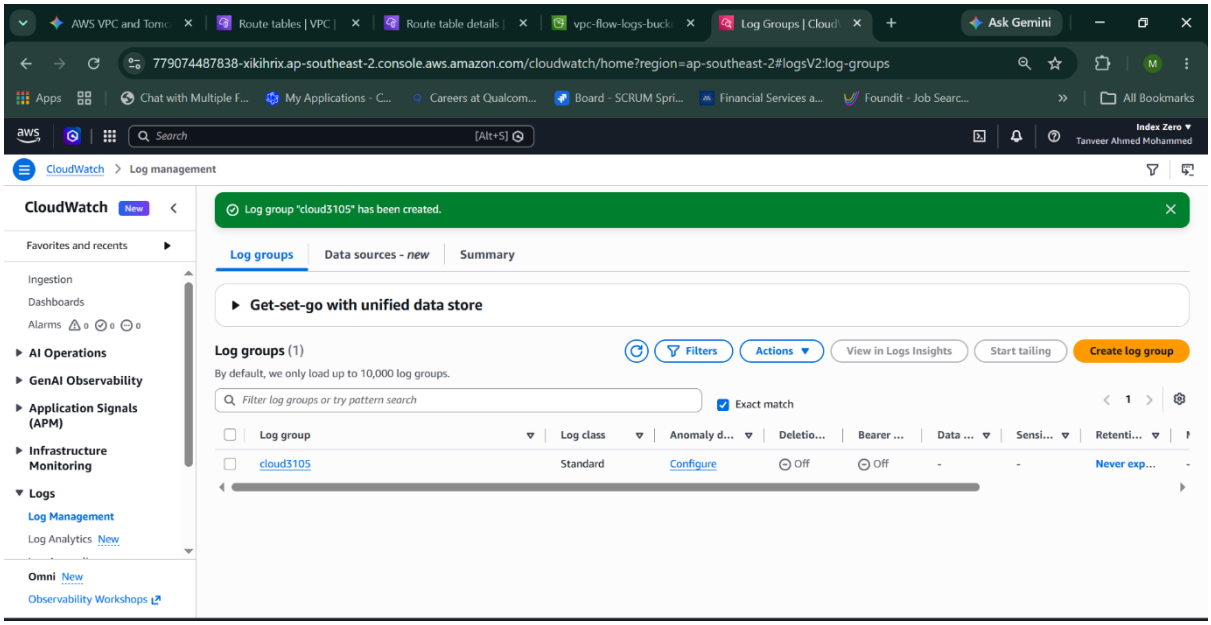


10) Configure VPC flow logs and store the logs in S3 and CloudWatch.



Step 1: Create an S3 Bucket (For S3 Log Destination)

1. Navigate to the S3 Console and click Create bucket.
2. Bucket name: Enter a globally unique name (e.g., vpc-flow-logs-bucket-3105).
3. Region: Select the same region as your VPC.
4. Block Public Access: Keep *Block all public access* checked.
5. Click Create bucket.
    - *Verification:* The bucket appears in your S3 bucket list with status successfully created.

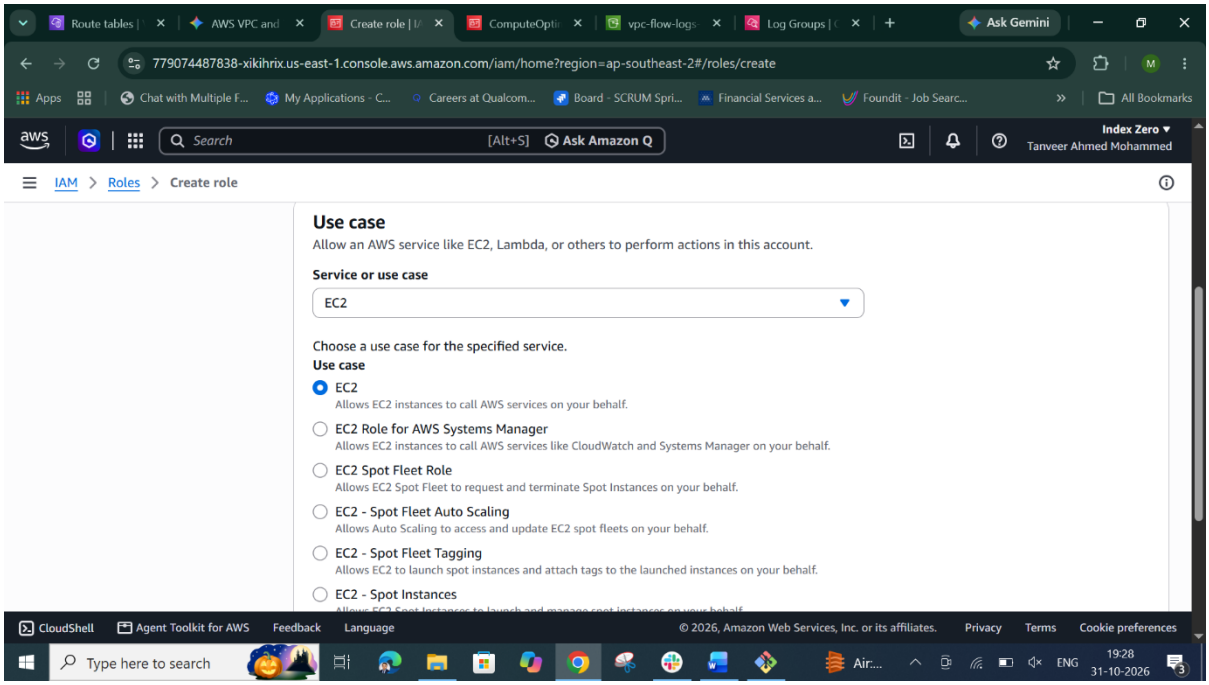


Step 2: Create a CloudWatch Log Group and IAM Role (For CloudWatch Destination)
Navigate to the CloudWatch Console, click Log groups in the left sidebar, and click Create log group.

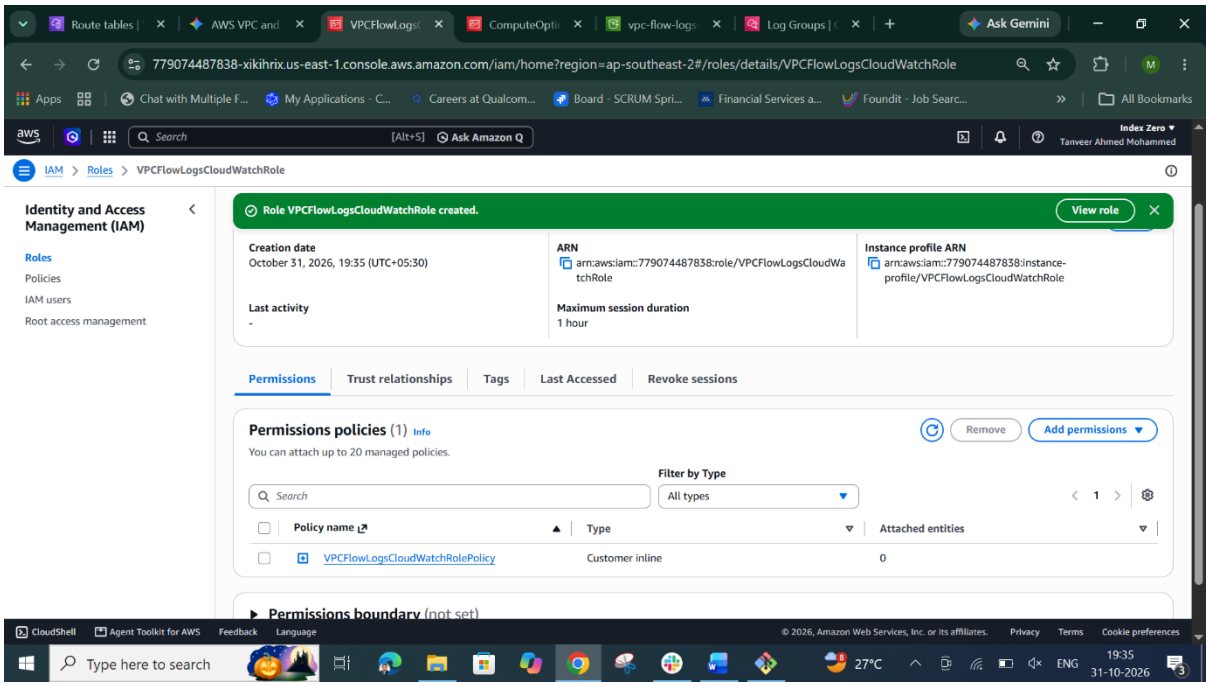
Log group name: /aws/vpc/flow-logs

Click Create.

Verification: The log group appears in the Log Groups list.



a


Route tables
AWS VPC and
VPCFlowLogs
ComputeOpti
vpc-flow-logs
Log Groups
Ask Gemini
779074487838-xikihrix.us-east-1.console.aws.amazon.com/iam/home?region=ap-southeast-2#/roles/details/VPCFlowLogsCloudWatchRole
Apps
Chat with Multiple F...
My Applications - C...
Careers at Qualcom...
Board - SCRUM Spri...
Financial Services a...
Foundit - Job Searc...
All Bookmarks
Search
[Alt+S]
Ask Amazon Q
Index Zero
Tanveer Ahmed Mohammed
IAM > Roles > VPCFlowLogsCloudWatchRole
Identity and Access Management (IAM)
Roles
Policies
IAM users
Root access management
Role VPCFlowLogsCloudWatchRole created.
View role
Creation date
October 31, 2026, 19:35 (UTC+05:30)
ARN
arn:aws:iam::779074487838:role/VPCFlowLogsCloudWatchRole
Instance profile ARN
arn:aws:iam::779074487838:instance-profile/VPCFlowLogsCloudWatchRole
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (1) Info
You can attach up to 20 managed policies.
Remove
Add permissions
Filter by Type
Search
All types
Policy name
Type
Attached entities
VPCFlowLogsCloudWatchRolePolicy
Customer inline
0
Permissions boundary (not set)
CloudShell
Agent Toolkit for AWS
Feedback
Language
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences
Type here to search
27°C
ENG
19:35
31-10-2026